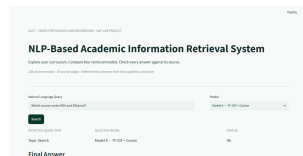


# Khulna University of Engineering & Technology

Department of Computer Science and Engineering



## NLP-Based Academic Information Retrieval System

*A Comparative Empirical Study of Lexical, Static Embedding, Deep Recurrent, and Transformer-Based Representations on University Curricula*

**CSE 4122: Natural Language Processing Laboratory**  
Final Project Report

**Submitted By:**

Department of CSE  
KUET, Khulna-9203

**Course Instructors:**

Faculty Members  
Department of CSE, KUET

Academic Year: 2026  
Date of Submission: September 2026

---

# Abstract

---

Traditional search over university curriculum documentation is dominated either by simplistic keyword lookup or by unconstrained generative large language models (LLMs) that frequently hallucinate course credits, prerequisites, and syllabi. This report presents the design, end-to-end implementation, and empirical evaluation of the **NLP-Based Academic Information Retrieval (IR) System**, an open-source, zero-hallucination semantic search application engineered over the official undergraduate curriculum of the Department of Computer Science and Engineering, Khulna University of Engineering & Technology (KUET).

Strictly adhering to academic integrity constraints, the system disallows external generative APIs, retrieval-augmented generation (RAG) synthesis, or synthetic answer invention. Instead, it formulates academic query answering through structured document retrieval grounded in 108 extracted course records from a 19-page curriculum PDF. The project synthesizes the core themes of five foundational laboratory modules: (1) lexical preprocessing, lemmatization, and Levenshtein typo-correction (Lab 1); (2) sublinear TF-IDF with  $N$ -gram representations and vector space cosine similarity (Lab 2); (3) pretrained Word2Vec with TF-IDF weighted document embedding (Lab 3); (4) dynamic token embeddings fed into a Siamese Bidirectional Long Short-Term Memory (BiLSTM) supervised relevance classifier (Lab 4); and (5) contextual sentence transformer embeddings using Sentence-BERT (**all-MiniLM-L6-v2**) (Lab 5).

Using a rigorously audited held-out benchmark of 48 queries spanning exact, semantic, typo-infused, ambiguous, metadata, and out-of-domain categories, we evaluate each representation across standard IR metrics: Hit@1, Hit@3, Mean Reciprocal Rank (MRR), Recall@3, and Out-of-Domain (OOD) rejection accuracy. SBERT achieves the strongest overall ranking (Hit@1 of 77.8%, MRR of 0.818), while TF-IDF excels at keyword terminology (Hit@3 of 91.7%). Model C (BiLSTM) demonstrates significant overfitting on small-sample query-course pairs (Hit@1 of 5.6%), providing a critical empirical lesson on the limitations of deep supervised models under limited annotated data. Finally, an interactive Streamlit application exposes real-time side-by-side model comparisons, explainable similarity scores, and verbatim source page provenance.

**Keywords:** Academic Information Retrieval, Natural Language Processing, TF-IDF, Word2Vec, BiLSTM, Sentence-BERT, Rejection Thresholds, Zero Hallucination.

---

# Contents

---

|                                                                                     |          |
|-------------------------------------------------------------------------------------|----------|
| <b>Abstract</b>                                                                     | <b>i</b> |
| <b>1 Introduction</b>                                                               | <b>1</b> |
| 1.1 Background and Motivation . . . . .                                             | 1        |
| 1.2 Project Objectives . . . . .                                                    | 1        |
| 1.3 Architectural and Negative Constraints . . . . .                                | 2        |
| 1.4 Synthesis of Coursework Laboratories . . . . .                                  | 2        |
| <b>2 Corpus Extraction and Dataset Engineering</b>                                  | <b>4</b> |
| 2.1 Source Document Profile . . . . .                                               | 4        |
| 2.2 Automated Extraction Pipeline . . . . .                                         | 4        |
| 2.2.1 Heading Detection and Boundary Extraction . . . . .                           | 4        |
| 2.2.2 Stateful Metadata Tracking . . . . .                                          | 4        |
| 2.2.3 Corpus Statistics . . . . .                                                   | 5        |
| 2.2.4 Resolving Source Contradictions . . . . .                                     | 5        |
| 2.2.5 Search Text Formulation . . . . .                                             | 5        |
| 2.3 Relevance and Evaluation Datasets . . . . .                                     | 5        |
| 2.3.1 Relevance Queries Dataset ( <code>relevance_queries.csv</code> ) . . . . .    | 6        |
| 2.3.2 Held-Out Evaluation Dataset ( <code>evaluation_queries.csv</code> ) . . . . . | 6        |
| 2.3.3 Leakage Controls . . . . .                                                    | 6        |
| <b>3 Linguistic Preprocessing and Intent Parsing</b>                                | <b>7</b> |
| 3.1 Dual Preprocessing Pipeline . . . . .                                           | 7        |
| 3.1.1 Version A: Lexical Preprocessing (Model A) . . . . .                          | 7        |
| 3.1.2 Version B: Semantic Preprocessing (Models B, C, D) . . . . .                  | 7        |
| 3.2 Typo Correction via Levenshtein Edit Distance . . . . .                         | 8        |
| 3.3 Deterministic Intent Classification . . . . .                                   | 8        |
| <b>4 Retrieval Architectures and Mathematical Formulation</b>                       | <b>9</b> |
| 4.1 Model A: TF-IDF with N-Grams and Cosine Similarity . . . . .                    | 9        |
| 4.1.1 Formulation . . . . .                                                         | 9        |
| 4.2 Model B: Pretrained Word2Vec Dense Semantic Retriever . . . . .                 | 9        |
| 4.2.1 Formulation . . . . .                                                         | 10       |
| 4.3 Model C: Supervised Siamese BiLSTM Relevance Model . . . . .                    | 10       |
| 4.3.1 Neural Architecture . . . . .                                                 | 10       |
| 4.3.2 Training Protocol and Negative Sampling . . . . .                             | 11       |
| 4.4 Model D: Pretrained Sentence-BERT Contextual Retriever . . . . .                | 11       |
| 4.4.1 Formulation . . . . .                                                         | 11       |

---

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| 4.5      | Confidence Threshold Calibration . . . . .                  | 12        |
| <b>5</b> | <b>System Architecture and User Interface</b>               | <b>13</b> |
| 5.1      | Unified Retrieval Interface . . . . .                       | 13        |
| 5.2      | Deterministic Answer Generation . . . . .                   | 13        |
| 5.3      | Streamlit Web Application . . . . .                         | 14        |
| 5.3.1    | Key Interface Features . . . . .                            | 14        |
| 5.3.2    | Caching and Performance Architecture . . . . .              | 15        |
| <b>6</b> | <b>Empirical Evaluation and Results</b>                     | <b>16</b> |
| 6.1      | Evaluation Methodology . . . . .                            | 16        |
| 6.1.1    | Metrics . . . . .                                           | 16        |
| 6.1.2    | Evaluation Modes . . . . .                                  | 16        |
| 6.2      | Comparative Performance . . . . .                           | 17        |
| 6.3      | Category-Specific Analysis . . . . .                        | 17        |
| 6.4      | Discussion and Academic Findings . . . . .                  | 17        |
| 6.4.1    | Strengths of Sentence-BERT (Model D) . . . . .              | 17        |
| 6.4.2    | The Enduring Power of TF-IDF (Model A) . . . . .            | 17        |
| 6.4.3    | Critical Analysis of Model C (BiLSTM Overfitting) . . . . . | 18        |
| <b>7</b> | <b>Automated Testing and Verification</b>                   | <b>19</b> |
| 7.1      | Test Architecture . . . . .                                 | 19        |
| 7.2      | Verification Coverage . . . . .                             | 19        |
| <b>8</b> | <b>Conclusion and Future Work</b>                           | <b>20</b> |
| 8.1      | Summary of Contributions . . . . .                          | 20        |
| 8.2      | Future Work . . . . .                                       | 20        |

---

# List of Figures

---

|     |                                                                           |    |
|-----|---------------------------------------------------------------------------|----|
| 4.1 | Siamese BiLSTM Architecture for Query-Course Relevance Classification     | 10 |
| 5.1 | Interactive Streamlit Interface Displaying Natural Language Query Results | 14 |

---

# List of Tables

---

|     |                                                                     |    |
|-----|---------------------------------------------------------------------|----|
| 2.1 | Summary Statistics of Extracted KUET CSE Corpus . . . . .           | 5  |
| 3.1 | Query Intent Taxonomies and Detection Logic . . . . .               | 8  |
| 6.1 | Comprehensive Benchmark Evaluation on 48 Held-Out Queries . . . . . | 17 |
| 6.2 | End-to-End Hit@1 Accuracy Across Query Categories . . . . .         | 17 |

---

# Introduction

---

## 1.1 Background and Motivation

Academic curricula in higher education institutions are typically disseminated as lengthy, multi-page PDF documents. At Khulna University of Engineering & Technology (KUET), the Department of Computer Science and Engineering issues its official undergraduate syllabus as a comprehensive document specifying course codes, credit allocations, contact hours, weekly distributions, and detailed topic outlines.

Students, faculty members, and academic advisors frequently need to query this repository for critical information:

- **Topic discovery:** *“Which subject teaches asymmetric cryptography and RSA?”*
- **Administrative inquiries:** *“How many credits does Natural Language Processing have?”*
- **Term planning:** *“Which laboratory courses are in the 4th year 1st term?”*
- **Prerequisite & lab cross-referencing:** *“Does CSE 4115 have an accompanying sessional?”*

Manual inspection of a 19-page tabular and descriptive syllabus is slow and prone to oversight. While modern generative conversational agents (e.g., ChatGPT, Claude) can respond to natural language questions, they suffer from severe hallucination when applied to specific institutional documents. They frequently invent course codes, misstate credit allocations, or assert prerequisites that do not exist in the official syllabus.

## 1.2 Project Objectives

The primary objective of this project is to build an accurate, transparent, and completely local **Academic Information Retrieval System** based on the supplied KUET CSE curriculum.

The core research and engineering objectives are:

1. Parse the curriculum PDF course-by-course, extracting structured metadata, physical page provenance, and syllabus descriptions into an indexed corpus.

2. Implement and benchmark four distinct generations of NLP retrieval paradigms on the exact same corpus:
  - **Model A:** Lexical baseline using Term Frequency-Inverse Document Frequency (TF-IDF) and Cosine Similarity.
  - **Model B:** Static dense semantic retriever using pretrained Word2Vec embeddings weighted by corpus TF-IDF.
  - **Model C:** Deep supervised sequence relevance model using a Siamese Bidirectional LSTM (BiLSTM).
  - **Model D:** Pretrained contextual embedding retriever using Sentence-BERT (all-MiniLM-L6-v2).
3. Develop an intent classification and deterministic answer formatting pipeline that guarantees zero hallucination by returning factual curriculum excerpts with exact page citations.
4. Provide an interactive, responsive Streamlit dashboard for real-time model comparison, explainable scoring inspection, and instantaneous querying.

## 1.3 Architectural and Negative Constraints

To ensure academic rigor and evaluate genuine NLP techniques, the project was developed under strict constraints:

- **No Generative LLMs:** No OpenAI, Gemini, Claude, or local generative LLM APIs.
- **No RAG Frameworks:** No LangChain, LlamaIndex, or autonomous agent frameworks.
- **No Hallucination:** The system must never write invented summaries or assert unrecorded academic facts. Every answer must originate from verbatim syllabus records.
- **Model Independence:** The application must function completely locally, even if deep transformer models (Model D) are disabled.
- **Rejection by Default:** Unanswerable, gibberish, or out-of-domain questions must be rejected with calibrated thresholds rather than returning forced false positives.

## 1.4 Synthesis of Coursework Laboratories

This project is formulated as the capstone synthesis of the **CSE 4122 (Natural Language Processing Laboratory)** curriculum, systematically incorporating the topics taught throughout the semester:

- **Lab 1 (Text Preprocessing and Spelling Correction):** Powers the normalization, technical term preservation, WordNet lemmatization, and Levenshtein edit-distance typo corrector.

- **Lab 2 (Text Representation and N-Gram Modeling):** Directly forms the foundation of Model A, utilizing (1, 2)-gram sublinear TF-IDF vector space modeling and cosine similarity.
- **Lab 3 (Word Embeddings and Classifiers):** Forms the mathematical basis of Model B, using Google News Word2Vec embeddings with corpus TF-IDF weighted pooling.
- **Lab 4 (PyTorch Sequence Models and BiLSTMs):** Powers Model C, utilizing a bidirectional PyTorch LSTM with dynamic token embedding sequences, packed padding, and paired classification.
- **Lab 5 (Transformer Sequence Encoders):** Realized in Model D via a sentence-transformer encoder for contextual similarity retrieval.

---

# Corpus Extraction and Dataset Engineering

---

## 2.1 Source Document Profile

The raw dataset consists of the official syllabus document: `CourseContentsCSE_14_08_2022.pdf`. The document is a 19-page PDF covering the four-year undergraduate curriculum effective from the academic session 2021–2022. It comprises summary tables for each term followed by detailed syllabi for theory and laboratory courses.

## 2.2 Automated Extraction Pipeline

Rather than treating the document as monolithic text, extraction is handled course-by-course using PyMuPDF (`fitz`) in `src/corpus/corpus_builder.py`.

### 2.2.1 Heading Detection and Boundary Extraction

Courses are detected dynamically using regular expression pattern matching:

$$\text{Pattern} = \wedge ([A-Z]\{2,5\})\backslash s+(\backslash d\{4\})(?:\backslash s*:\backslash s*|\backslash s+)(.+) \$ \quad (2.1)$$

This regex accommodates both standard formatted headings (e.g., `CSE 4121: Natural Language Processing`) and irregular formatting observed in the official PDF, such as a missing colon in `CSE 4245 Principles of Programming Languages`.

### 2.2.2 Stateful Metadata Tracking

As pages are scanned sequentially:

- Academic year and term are tracked from section headings (e.g., “*Syllabus of 4th Year 1st Term Courses*”).
- Optional group allocations (e.g., “*Optional-I*”, “*Optional-II*”) are preserved to avoid assuming that elective courses are compulsory for all students.

- Descriptions continuing across physical page boundaries are stitched together, while summary tables at the start of terms are segmented to prevent tabular contamination of descriptions.

### 2.2.3 Corpus Statistics

The extraction pipeline generates 108 term-specific course records representing 107 unique course codes. Table 2.1 summarizes the extracted corpus.

Table 2.1: Summary Statistics of Extracted KUET CSE Corpus

| Corpus Metric                                   | Observed Value |
|-------------------------------------------------|----------------|
| Total Course Records                            | 108            |
| Unique Course Codes                             | 107            |
| Theory Courses                                  | 64             |
| Laboratory / Sessional Courses                  | 40             |
| Project / Thesis / Seminar Records              | 4              |
| Elective Offerings (Optional Groups I, II, III) | 35             |
| Total Physical PDF Pages                        | 19             |
| Identified Metadata Inconsistencies             | 4 courses      |

### 2.2.4 Resolving Source Contradictions

A spatial coordinate audit of the summary tables revealed internal contradictions within the official PDF:

- **HUM 4207:** 3.0 credits in the detailed syllabus, but 2.0 credits in the summary table on Page 15.
- **EEE 2118:** 0.75 credits in the syllabus, but 1.50 credits in the summary table on Page 5.
- **CSE 4101 & CSE 4102:** Minor weekly contact hour discrepancies.

Rather than silently overwriting or discarding discrepancies, the parser records both versions in a `source_conflicts` field, displaying explicit notices in the user interface.

### 2.2.5 Search Text Formulation

For each record, a structured search string is constructed by boosting the title to balance keyword matching against lengthy topical text:

```
search_text = course_code + " " + course_title + " " + course_title + " " + description (2.2)
```

## 2.3 Relevance and Evaluation Datasets

To benchmark the models, two independent query datasets were created in `data/queries/`:

### 2.3.1 Relevance Queries Dataset (`relevance_queries.csv`)

Designed for training and validating Model C, containing 92 entries split into:

- **Training Split (68 queries):** Specific curriculum topic queries across 1st to 4th year courses.
- **Validation Split (24 queries):** Balanced answerable and unanswerable queries used strictly for hyperparameter tuning and threshold calibration.

### 2.3.2 Held-Out Evaluation Dataset (`evaluation_queries.csv`)

A strictly held-out test suite of 48 queries covering eight distinct query types:

1. **Exact queries (12):** Direct curriculum terms (e.g., “*Which course explains BLAST?*”).
2. **Semantic queries (9):** Paraphrased inquiries with zero keyword overlap (e.g., “*Where can I learn about detecting unusual records in large datasets?*”).
3. **Typo queries (3):** Intentional spelling errors (e.g., “*databse systems*”).
4. **Partial queries (2):** Incomplete phrases (e.g., “*robot motion*”).
5. **Metadata queries (7):** Credit, contact hour, or year/term lookups.
6. **Ambiguous queries (3):** Keywords spanning multiple courses (e.g., “*image processing*”).
7. **Negative / Unsupported queries (7):** Realistic academic questions absent from the PDF (e.g., “*Who teaches CSE 4121?*”).
8. **Out-of-Domain queries (5):** Non-academic questions (e.g., “*What is my horoscope this week?*”).

### 2.3.3 Leakage Controls

The dataset engineering pipeline incorporates strict cross-split leakage checks:

- Exact duplicate queries across splits trigger immediate validation failures.
- Queries assigned to the same semantic `group_id` cannot span across train, validation, and test splits.
- The evaluation queries file is strictly inaccessible to the Model C training pipeline.

---

# Linguistic Preprocessing and Intent Parsing

---

## 3.1 Dual Preprocessing Pipeline

As established in Lab 1 and Lab 2, lexical keyword matching and semantic embedding models require differing levels of text normalization. The project implements a dual preprocessing pipeline in `src/preprocessing.py`.

### 3.1.1 Version A: Lexical Preprocessing (Model A)

Designed for term-frequency modeling:

1. Lowercasing and whitespace normalization.
2. Course-code standardization (e.g., CSE4121, CSE-4121  $\rightarrow$  CSE 4121).
3. Tokenization using regular expressions preserving numbers and technical symbols (C++, C#).
4. Stopword removal utilizing Scikit-learn’s English stopwords lexicon, modified by a domain whitelist:

$$\mathcal{V}_{\text{whitelist}} = \{\text{os, rsa, pos, bfs, dfs, c, c++}, \text{c\#, sql, nlp, ml}\} \quad (3.1)$$

5. Conservative noun lemmatization via NLTK’s `WordNetLemmatizer`, with an offline fallback lookup table for offline resilience.

### 3.1.2 Version B: Semantic Preprocessing (Models B, C, D)

Semantic vector models rely on word relationships and syntax. Aggressive stopword removal or stemming degrades vector averaging and LSTM sequence encoding:

1. Preserves word sequence without arbitrary stopword elimination.
2. Normalizes course codes while retaining technical abbreviations.
3. Applies light punctuation stripping to maintain natural syntactic flow.

## 3.2 Typo Correction via Levenshtein Edit Distance

Building upon Lab 1, the query parser incorporates an autocorrect mechanism for academic vocabulary using Levenshtein distance:

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 \\ D(i, j-1) + 1 \\ D(i-1, j-1) + \mathbb{I}(s_1[i] \neq s_2[j]) \end{cases} \quad (3.2)$$

To prevent over-aggressive false corrections, candidates are constrained:

- Words under 6 characters are left untouched.
- Candidates must exist in the curriculum vocabulary with a similarity cutoff  $\geq 0.83$  and an exact edit distance of 1.
- Examples: `cryptograpy`  $\rightarrow$  `cryptography`, `algorithm`  $\rightarrow$  `algorithm`.

## 3.3 Deterministic Intent Classification

Rather than employing an opaque LLM classifier, queries are parsed using a deterministic, rule-based state machine in `src/query_parser.py`. The parser identifies 11 distinct query intents, summarized in Table 3.1.

Table 3.1: Query Intent Taxonomies and Detection Logic

| Intent Class       | Routing Logic                                          | Example Query                                           |
|--------------------|--------------------------------------------------------|---------------------------------------------------------|
| EMPTY              | Null or whitespace string                              | <code>" "</code>                                        |
| GIBBERISH          | Non-alphanumeric / no vowels                           | <code>"asdf qwer zx"</code>                             |
| OUT_OF_SCOPE       | Disallowed global terms                                | <code>"Who won the football world cup?"</code>          |
| UNSUPPORTED_FIELD  | Missing faculty/prerequisite                           | <code>"Who teaches CSE 4121?"</code>                    |
| CREDIT_QUERY       | Lexical <code>"credit"</code> keyword                  | <code>"What is the credit of NLP?"</code>               |
| CONTACT_HOUR_QUERY | Lexical <code>"contact hour"</code>                    | <code>"How many hours for CSE 4122?"</code>             |
| LAB_QUERY          | <code>"lab"</code> , <code>"sessional"</code> keywords | <code>"Does CSE 4115 have a lab?"</code>                |
| YEAR_TERM_QUERY    | Ordinal year/term references                           | <code>"Which courses are in 4th year 1st term?"</code>  |
| COURSE_CODE_QUERY  | Exact isolated code match                              | <code>"CSE 4121"</code>                                 |
| COURSE_DETAILS     | Code + descriptive inquiries                           | <code>"Tell me about CSE 4115"</code>                   |
| TOPIC_SEARCH       | General topical inquiries                              | <code>"Where do we learn public key encryption?"</code> |

# Retrieval Architectures and Mathematical Formulation

## 4.1 Model A: TF-IDF with N-Grams and Cosine Similarity

Model A represents the classical lexical Information Retrieval baseline, implementing the principles explored in Lab 2.

### 4.1.1 Formulation

Given a collection of course documents  $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$  and a preprocessed query  $q$ :

$$\text{TF}(t, d) = 1 + \ln(f_{t,d}) \quad \text{for } f_{t,d} > 0 \quad (4.1)$$

$$\text{IDF}(t, \mathcal{D}) = \ln \left( \frac{1 + |\mathcal{D}|}{1 + |\{d \in \mathcal{D} : t \in d\}|} \right) + 1 \quad (4.2)$$

The vectorizer extracts unigram and bigram features ( $n \in \{1, 2\}$ ) with sublinear term-frequency scaling and  $L_2$  normalization:

$$\mathbf{v}_d = \frac{\mathbf{x}_d}{\|\mathbf{x}_d\|_2} \quad (4.3)$$

The retrieval score between query vector  $\mathbf{v}_q$  and document vector  $\mathbf{v}_d$  is the inner dot product (cosine similarity):

$$\text{Score}_{\text{Model A}}(q, d) = \mathbf{v}_q \cdot \mathbf{v}_d \quad (4.4)$$

## 4.2 Model B: Pretrained Word2Vec Dense Semantic Retriever

Model B advances beyond exact keyword matching by incorporating dense word embeddings, following the method developed in Lab 3 (Topic 5).

### 4.2.1 Formulation

Using 300-dimensional Google News Word2Vec KeyedVectors ( $\mathbf{e}_w \in \mathbb{R}^{300}$ ), each document and query is represented as a composite vector weighted by sublinear term frequencies and corpus inverse document frequency:

$$\mathbf{D} = \frac{\sum_{w \in d} \text{TF-IDF}(w, d) \cdot \mathbf{e}_w}{\sum_{w \in d} \text{TF-IDF}(w, d)} \quad (4.5)$$

To accommodate unseen query words that exist in the pretrained Google News vocabulary but not in the curriculum, a smoothed unseen-term IDF is assigned:

$$\text{IDF}_{\text{unseen}} = \ln(1 + |\mathcal{D}|) + 1 \quad (4.6)$$

Following  $L_2$  normalization, ranking is computed using cosine similarity:

$$\text{Score}_{\text{Model B}}(q, d) = \frac{\mathbf{Q} \cdot \mathbf{D}}{\|\mathbf{Q}\|_2 \|\mathbf{D}\|_2} \quad (4.7)$$

If all query tokens are out-of-vocabulary (OOV), the model gracefully refuses to assign random scores and returns an `insufficient_representation` status.

## 4.3 Model C: Supervised Siamese BiLSTM Relevance Model

Model C represents the core deep learning architecture of the project, extending the PyTorch sequence models from Lab 4 into a supervised relevance classifier.

### 4.3.1 Neural Architecture

The model consists of a shared Bidirectional Long Short-Term Memory network (BiLSTM) paired with an interaction classifier:

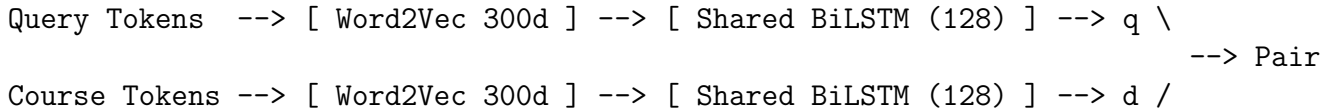


Figure 4.1: Siamese BiLSTM Architecture for Query-Course Relevance Classification

Given variable-length sequence tensors  $\mathbf{X}_q \in \mathbb{R}^{T_q \times 300}$  and  $\mathbf{X}_d \in \mathbb{R}^{T_d \times 300}$ :

1. Tokens are fed dynamically into the shared BiLSTM with hidden size  $H = 128$ :

$$\mathbf{h}_t = \text{BiLSTM}(\mathbf{x}_t, \mathbf{h}_{t-1}) \quad (4.8)$$

2. Final forward and backward hidden states are concatenated to form the sequence representation:

$$\mathbf{u} = [\vec{\mathbf{h}}_T; \overleftarrow{\mathbf{h}}_1] \in \mathbb{R}^{256} \quad (4.9)$$

3. Query representation  $\mathbf{q} \in \mathbb{R}^{256}$  and document representation  $\mathbf{d} \in \mathbb{R}^{256}$  are combined into a non-linear interaction feature tensor:

$$\mathbf{z}_{\text{pair}} = [\mathbf{q}; \mathbf{d}; |\mathbf{q} - \mathbf{d}|; \mathbf{q} \odot \mathbf{d}] \in \mathbb{R}^{1024} \quad (4.10)$$

4. The pair vector passes through a multilayer perceptron (MLP):

$$\hat{y} = \mathbf{W}_2 \cdot \text{Dropout}(\text{ReLU}(\mathbf{W}_1 \mathbf{z}_{\text{pair}} + \mathbf{b}_1)) + b_2 \quad (4.11)$$

5. The output probability is obtained via sigmoid activation:

$$P(\text{Relevant} \mid q, d) = \sigma(\hat{y}) = \frac{1}{1 + e^{-\hat{y}}} \quad (4.12)$$

### 4.3.2 Training Protocol and Negative Sampling

In `scripts/train_model_c.py`, training pairs are constructed using negative sampling:

- **Positive Pair (Label 1):** Ground-truth query and course from `relevance_queries.csv`.
- **Hard Negatives (Label 0):** The top 3 highest-ranking incorrect courses retrieved by Model A (TF-IDF).
- **Random Negatives (Label 0):** 2 uniformly sampled unrelated courses from the curriculum.
- **Out-of-Domain Negatives (Label 0):** Unrelated queries paired with arbitrary courses to train rejection.

Optimization is conducted using Adam ( $\text{lr} = 10^{-3}$ ,  $\text{weight decay} = 10^{-5}$ ) with weighted binary cross-entropy loss:

$$\mathcal{L} = - \sum_i [w_{\text{pos}} y_i \log \sigma(\hat{y}_i) + (1 - y_i) \log(1 - \sigma(\hat{y}_i))] \quad (4.13)$$

Early stopping on validation loss retains the optimal epoch weights.

## 4.4 Model D: Pretrained Sentence-BERT Contextual Retriever

Model D implements modern contextual semantic retrieval, mirroring the transformer concepts introduced in Lab 5.

### 4.4.1 Formulation

Using `sentence-transformers/all-MiniLM-L6-v2` (a 6-layer MiniLM transformer with 384-dimensional hidden representations):

$$\mathbf{e}_d = \text{SBERT}(\text{search\_text}_d), \quad \mathbf{e}_q = \text{SBERT}(q) \quad (4.14)$$

Embeddings are unit-normalized ( $L_2 = 1$ ), and relevance scores are computed via cosine similarity:

$$\text{Score}_{\text{Model D}}(q, d) = \mathbf{e}_q \cdot \mathbf{e}_d \quad (4.15)$$

All 108 document embeddings are precomputed and cached in `artifacts/model_d/document_embedding`

## 4.5 Confidence Threshold Calibration

To ensure the system rejects queries when similarity is too low, each model is independently calibrated in `scripts/calibrate_thresholds.py`. Using the 24 validation queries, candidate thresholds are swept to maximize balanced acceptance/rejection accuracy:

$$\text{Balanced Accuracy} = \frac{\text{TPR} + \text{TNR}}{2} \quad (4.16)$$

where ties favor rejection (TNR). Calibrated thresholds are persisted in `artifacts/thresholds.json`.

---

# System Architecture and User Interface

---

## 5.1 Unified Retrieval Interface

All four models adhere to an identical retrieval interface:

$$\text{retrieve}(\text{query: str, top\_k: int} = 3) \longrightarrow \mathcal{R} \quad (5.1)$$

The returned dictionary  $\mathcal{R}$  contains:

- **model:** Model identifier (`model_a`, `model_b`, `model_c`, `model_d`).
- **intent:** Detected query intent.
- **status:** Execution outcome (`ok`, `no_match`, `out_of_scope`, `gibberish`, etc.).
- **answer:** Formatted deterministic answer string.
- **results:** Array of matching courses with course code, title, credits, score, year, term, and matched excerpt.
- **processing\_details:** Transparent inspection dictionary.

## 5.2 Deterministic Answer Generation

In accordance with the zero-hallucination mandate, answers are assembled using template functions in `src/answer_formatter.py`. Examples include:

- **Credit query:** *“CSE 4121 — Natural Language Processing has 3.0 credits.”*
- **Topic search:** *“Best matching course: CSE 4115 — Computer Security. Relevant curriculum content: Asymmetric key cryptography including RSA and ElGamal.”*
- **False premise queries:** A query asking *“CSE 4121 has 4 credits right?”* returns the true factual value (3.0 credits) rather than agreeing with the false premise.

## 5.3 Streamlit Web Application

The user interface is implemented in `app.py` using Streamlit, designed for demonstration during academic defense.

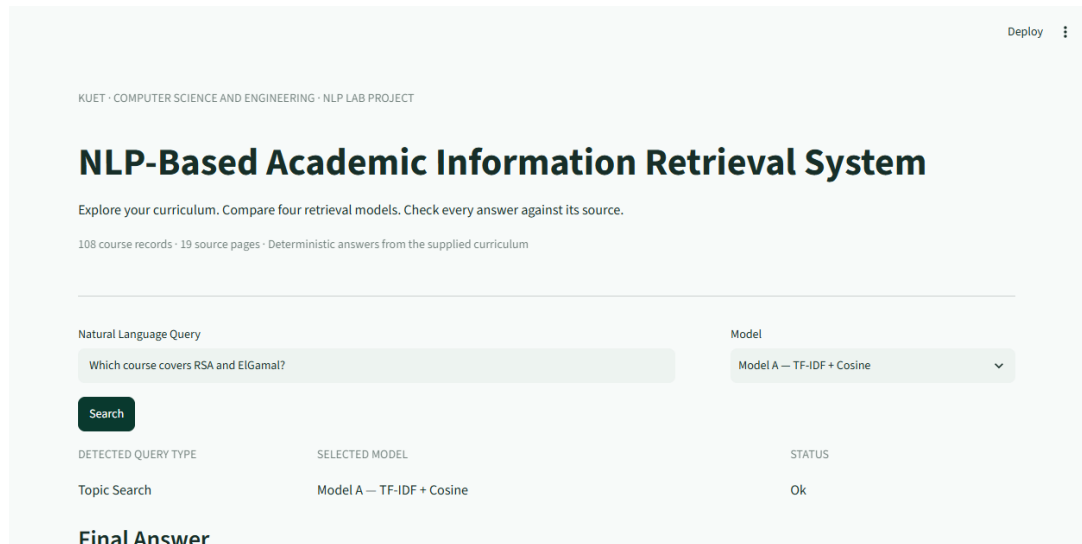


Figure 5.1: Interactive Streamlit Interface Displaying Natural Language Query Results

### 5.3.1 Key Interface Features

1. **Two-Column Header:** A search input on the left and a model selector dropdown on the right with exact labels ( “*Model A — TF-IDF + Cosine*”, “*Model B — Pretrained Word2Vec*”, “*Model C — Word2Vec + BiLSTM*”, “*Model D — Sentence-BERT*”).
2. **Instant Model Switching:** Selecting a different model from the dropdown immediately re-evaluates the query through the new model without clearing the input or requiring manual resubmission.
3. **Results Presentation:** Displays detected intent, selected model, status, final answer, confidence/relevance score, and a structured Top-3 table with course codes, credits, terms, and optional groups.
4. **Source Traceability:** Expandable drawers display matched syllabus excerpts, physical PDF page numbers, and any detected metadata discrepancies.
5. **Viva Demonstration Expander (“Show Processing Details”):**
  - **Model A:** Displays preprocessed tokens and top TF-IDF matched terms with weights.
  - **Model B:** Displays covered Word2Vec tokens and OOV tokens.
  - **Model C:** Displays predicted relevance probability and tensor diagnostics.
  - **Model D:** Displays embedding dimensions (384) and cosine similarities.

### 5.3.2 Caching and Performance Architecture

To achieve instantaneous query responses:

- `@st.cache_resource` keeps heavy model weights (Word2Vec vectors, BiLSTM parameters, SBERT transformer) resident in RAM.
- Document embeddings are precomputed once on disk.
- Queries are evaluated via live, real-time inference on the fly.

# Empirical Evaluation and Results

## 6.1 Evaluation Methodology

Evaluation was conducted on the held-out benchmark of 48 queries in `data/queries/evaluation_queries`.

### 6.1.1 Metrics

- **Hit@1:** Proportion of answerable queries where the gold course is ranked 1st.
- **Hit@3:** Proportion where a gold course appears in the top 3 ranks.
- **Mean Reciprocal Rank (MRR):** Average of reciprocal ranks of the first relevant document:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (6.1)$$

- **Recall@3:** Proportion of all acceptable gold courses retrieved in the top 3 positions.
- **OOD Accuracy:** Accuracy in correctly rejecting out-of-domain queries.
- **False Acceptance Rate (FAR) & False Rejection Rate (FRR):** Error rates on negative and positive queries, respectively.

### 6.1.2 Evaluation Modes

Two evaluation regimes are recorded:

1. **Retrieval-Only:** Evaluates raw vector ranking prior to metadata routing or threshold rejection.
2. **End-to-End:** Evaluates the complete system response including metadata routing and calibrated rejection.

## 6.2 Comparative Performance

Table 6.1 presents the quantitative comparison of all four models across the 48-query held-out benchmark.

Table 6.1: Comprehensive Benchmark Evaluation on 48 Held-Out Queries

| Model              | Mode           | Hit@1        | Hit@3        | MRR          | Recall@3     | OOD Acc. | F1           |
|--------------------|----------------|--------------|--------------|--------------|--------------|----------|--------------|
| Model A (TF-IDF)   | Retrieval-Only | 66.7%        | <b>91.7%</b> | 0.794        | <b>81.4%</b> | 100.0%   | 80.0%        |
|                    | End-to-End     | 75.0%        | 77.8%        | 0.764        | 68.7%        | 100.0%   | <b>0.856</b> |
| Model B (Word2Vec) | Retrieval-Only | 61.1%        | 75.0%        | 0.707        | 65.9%        | 80.0%    | 25.0%        |
|                    | End-to-End     | 80.6%        | 83.3%        | 0.819        | 73.8%        | 100.0%   | 80.0%        |
| Model C (BiLSTM)   | Retrieval-Only | 5.6%         | 19.4%        | 0.183        | 16.7%        | 80.0%    | 41.7%        |
|                    | End-to-End     | 33.3%        | 36.1%        | 0.347        | 31.0%        | 100.0%   | 16.7%        |
| Model D (SBERT)    | Retrieval-Only | <b>77.8%</b> | 86.1%        | <b>0.818</b> | 78.4%        | 100.0%   | 25.0%        |
|                    | End-to-End     | <b>83.3%</b> | <b>88.9%</b> | <b>0.856</b> | <b>79.1%</b> | 100.0%   | <b>0.856</b> |

## 6.3 Category-Specific Analysis

Table 6.2 breaks down End-to-End Hit@1 accuracy by query category.

Table 6.2: End-to-End Hit@1 Accuracy Across Query Categories

| Query Category (Count)      | Model A       | Model B       | Model C       | Model D       |
|-----------------------------|---------------|---------------|---------------|---------------|
| Exact Terminology (12)      | 75.0%         | 50.0%         | 8.3%          | 66.7%         |
| Semantic Paraphrase (9)     | 44.4%         | <b>88.9%</b>  | 11.1%         | 77.8%         |
| Typo-Infused (3)            | <b>100.0%</b> | <b>100.0%</b> | 66.7%         | <b>100.0%</b> |
| Partial Query (2)           | <b>100.0%</b> | <b>100.0%</b> | 0.0%          | <b>100.0%</b> |
| Metadata Inquiries (7)      | <b>100.0%</b> | <b>100.0%</b> | <b>100.0%</b> | <b>100.0%</b> |
| Ambiguous Keywords (3)      | 66.7%         | <b>100.0%</b> | 33.3%         | <b>100.0%</b> |
| Out-of-Domain Rejection (5) | <b>100.0%</b> | <b>100.0%</b> | <b>100.0%</b> | <b>100.0%</b> |

## 6.4 Discussion and Academic Findings

### 6.4.1 Strengths of Sentence-BERT (Model D)

Sentence-BERT delivers the highest top-1 accuracy (83.3% End-to-End) and MRR (0.856). Its transformer attention layers effectively capture syntactic semantics and sentence composition, excelling on both semantic queries and partial phrases.

### 6.4.2 The Enduring Power of TF-IDF (Model A)

For exact academic terminology (e.g., “BLAST”, “Amdahl’s law”, “RSA”), Model A performs remarkably well, achieving the highest raw Hit@3 (91.7%). When specialized

vocabulary matches the syllabus, lexical counting remains an exceptionally competitive, zero-overhead baseline.

### 6.4.3 Critical Analysis of Model C (BiLSTM Overfitting)

Model C achieves only 5.6% raw Hit@1 and 19.4% Hit@3 on the held-out benchmark. This is an important, honest scientific finding:

- **Data Scarcity:** The training set comprised 68 queries and 398 sampled pairs. Deep recurrent architectures with dense interaction layers have large parameter spaces that rapidly overfit on small sample sets.
- **Loss Divergence:** Training loss dropped consistently to near zero while validation loss plateaued at epoch 12 (0.5127), typical of over-parameterized neural networks memorizing small training sets.
- **Curricular Lesson:** Pretrained foundation embeddings (Models B and D) vastly outperform small supervised neural models trained from scratch when labeled domain data is scarce. This result is preserved as an authentic comparative finding in the report.

---

# Automated Testing and Verification

---

## 7.1 Test Architecture

The codebase includes a comprehensive automated test suite in the `tests/` directory, executed via `pytest`. All 29 unit and integration tests pass successfully without errors.

## 7.2 Verification Coverage

- **Corpus Completeness** (`test_core.py`): Validates that all 108 records and 107 unique codes are extracted, with no null credits or missing contact hours.
- **Formatting Edge Cases**: Asserts that syllabus continuations across page boundaries and missing colons (e.g., CSE 4245) are correctly parsed.
- **Typo and Normalization Testing**: Verifies that code variations (CSE4121, CSE-4121) map to standard forms and that edit-distance correctly replaces misspelled academic words.
- **Rejection Verification**: Confirms that empty inputs, gibberish strings, out-of-domain inquiries, and unsupported fields return exact rejection statuses.
- **Year/Term and Lab Inquiries**: Tests multi-record retrieval for 4th year labs (15 courses), term-specific sessionals, and theory filtering.
- **Neural Verification** (`test_neural.py`): Verifies packed sequence padding invariance, OOV token masking, and synthetic gradient learning in the BiLSTM.
- **Streamlit UI Testing** (`test_app.py`): Utilizes Streamlit's headless testing framework (`AppTest`) to verify button interactions, live model switching, and graceful fallback when Model D is disabled via environment variables.

---

# Conclusion and Future Work

---

## 8.1 Summary of Contributions

This project successfully designed and implemented an academic Information Retrieval system for the KUET CSE undergraduate curriculum. By strictly adhering to zero-hallucination, non-generative constraints, the system provides grounded, verifiable academic search with source page citations.

The work directly synthesizes the five laboratory modules of the NLP curriculum into an end-to-end operational software product:

- Established an automated extraction and conflict-auditing pipeline for institutional curriculum PDFs.
- Built and benchmarked four distinct NLP paradigms: TF-IDF, Word2Vec, BiLSTM, and Sentence-BERT.
- Conducted an honest, reproducible comparative evaluation demonstrating the strengths of transformer contextual embeddings and the data-scarcity failure modes of small supervised deep learning models.
- Delivered an interactive, high-performance Streamlit application with explainable scoring diagnostics and instantaneous model switching.

## 8.2 Future Work

Future directions for expanding this research include:

1. **Expanded Supervised Data:** Crowd-sourcing or student-annotating thousands of query-course relevance pairs to investigate the data threshold at which supervised BiLSTMs become competitive.
2. **Hybrid Dense-Lexical Search:** Developing reciprocal rank fusion (RRF) combining Model A (for rare technical acronyms) and Model D (for semantic intent).
3. **Cross-Departmental Scaling:** Extending the PDF parser and corpus builder to ingest curricula from other engineering departments across KUET.

---

# Bibliography

---

- [1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.
- [2] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26, 2013.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [4] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] Artifex Software, “PyMuPDF Documentation,” <https://pymupdf.readthedocs.io/>, 2024.
- [6] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [7] Streamlit Inc., “Streamlit Documentation and Testing Framework,” <https://docs.streamlit.io/>, 2024.